PalmSource®
Conference and Expo



Building International Applications

Ezekiel Sanborn de Asis
DTS Engineer, PalmSource, Inc.

A Brief History of Time

Palm OS® 3.1

- Japanese OS
- Text Manager
- International Manager
- Shift-JIS Character Encoding

Palm OS® 3.5

- Overlay Manager
- Custom Date/Time Formats
- TxtParamString

Palm OS® 4.0

- Locale Manager
- Locale Module
- New Time Zone Support
- Table Driven Text Manager

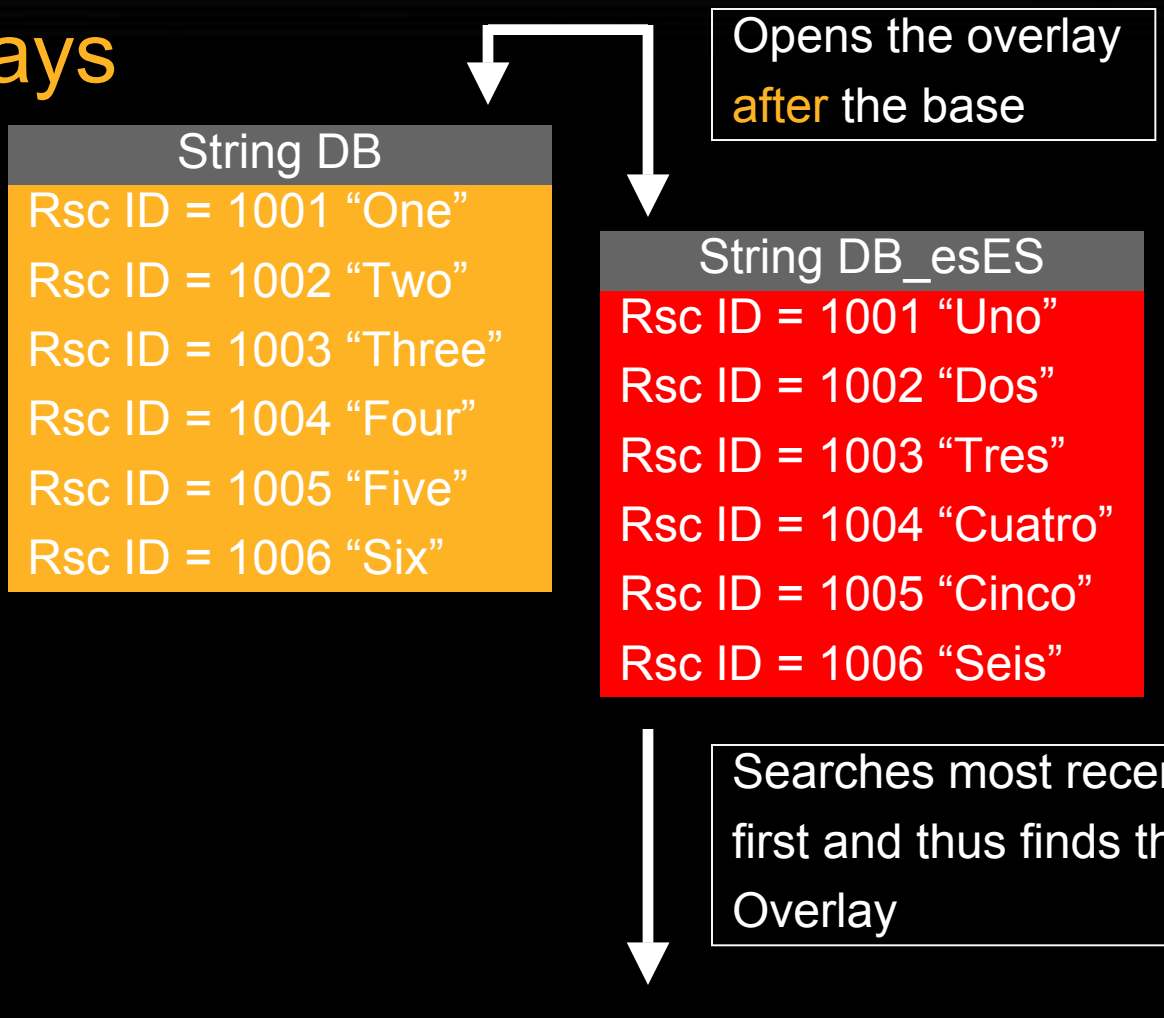
Text Manager (3.1 OS)

- Character Sizes, Attributes and Encodings
- Word Bounds and String Truncation
- Searching, Comparing and Transliterating Text
- Previous and Next Characters
- TxtFindString
- TxtParamString from 3.5 OS
- TxtCovertEncoding from 4.0 OS
- FntCharWidth, TxtCharWidth, FntWCharWidth
- Table Driven from 4.0 OS

Overlay Manager (3.5 OS)

- Separate Localized PRC
- One Base PRC can have many Overlay PRCs
- Overlay Manager automatically opens overlay PRC and substitutes localized overlay resources at runtime
  - You usually do not call the Overlay Manager
  - If overlay PRC for current locale exists, the Overlay Mgr will open the overlay after the base
  - You can specify not to use overlays, or call the Overlay Mgr to force the use of an overlay that does not match the current locale

Overlays

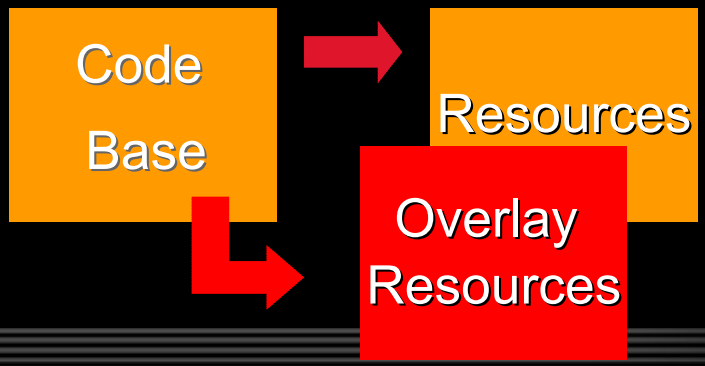



```
DmGetResource(StringRsc, 1003);
```

Overlays

- Name_InCN (example Name_enUS)
  - Use PRC2OVL -showlocales for list
- Overlay PRC type is ‘ovly’
- Overlay PRC creator is same as Base PRC
- ‘ovly’ =1000 that matches Base DB
- Matching Resource Ids, sizes and checksums





Date Time Formats

- Date Format in System Preferences
- DateTemplateToAscii from 3.5 OS
- Date Format moved out of ‘cnty’ resource in 4.0 OS (see Locale Manager)
- Time Zone support from 4.0 OS
  - Date Time Panel with Time Zone and DST on/off
  - Not automatic Daylight Savings Time (DST)
  - No automatic adjustment of appointments
  - Not a complete list of cities and Time Zones
- Be careful trying to get localized string values directly from system resources

Locale Module (4.0 OS)

- Resource DB specific to a character encoding defining OS behavior relative to that encoding
- Many locales can be based on 1 Locale Module
- Latin, Shift-JIS, GBK, Big-5 Locale Modules
- Contains
  - Data and Tables to drive Text Manager
  - Resources and Displays (example: Keyboard)
  - Fonts
  - Code to patch out OS routines
- The end result is that it is easier for us to add support for new locales

Locale Manager (4.0 OS)

- Provides a consistent method to get and set locale settings
  - Language, country, Date Format, Time Zone, etc.
  - See LocaleMgr.h for complete listing
  - **`LmGetLocaleSetting`**
  - **`LmLocaleToIndex`**
  - **`LmGetNumLocales`**
  - **`Use LmGlueXXX for older OS versions`**
- ‘cnty’ resource deprecated
- OmLocaleType replaced by LmLocaleType

Where the &%*! Am I?

- PrefGetPreference(prefLocale)—set by user
  - Value returned from prefTimeZoneCountry is different
- OmGetCurrentLocale()—overlay chooser
- OmGetSystemLocale()—sorting/searching rules
- Default Locale—fallback for current
- ROM Locale—survives a hard reset

Globalizing Your Code

- Follow the formats set by the user for Dates, Times, numbers, etc.
- Leave room for longer foreign text
- Prepare for other character encodings
- Consider double byte characters when manipulating, comparing or traversing strings
- Use templates for text generated at run time
- Prepare to be overlayed
- Make Base PRC resources locale-independent
- Take advantage of the PalmOSGlue library

Use Palm OS Glue

- Linkable Lib that provides backwards compatibility (new APIs work on older devices)
- Add to your project PalmOSGlue.lib from *Palm OS Support\CodeWarrior Libraries\Palm OS Glue*
- #include glue header files from *Palm OS Support\Incs\Libraries\PalmOSGlue*
- More info in PalmOSGlue Library section of OS Reference Part IV

User Defined Formats

- For Preferences that might be customized by the user (country, date format, etc.)
  - Use PrefGetPreference with appropriate SystemPreferencesChoice selector from Preferences.h
- Example

```
dateFormat=
    (DateFormatType)PrefGetPreference(prefDateFormat);
```

Locale Dependent Formats

- Use the Locale Manager for formats and values that are specific to a Locale
- Things like country names will come back in the language of the device
- Example: currency symbol

```
Char currencySymbol[kMaxCurrencySymbolLen+1];
UInt16 localeIndex;
UInt32 result = PrefGetPreference(prefLocale);
LmLocaleType* pUserLocale = (LmLocaleType*)&result;
LmLocaleToIndex(pUserLocale, &localeIndex);
LmGetLocaleSetting(localeIndex, lmChoiceCurrencySymbol,
  currencySymbol, sizeof(currencySymbol))
```

Character Encodings

Size Matters

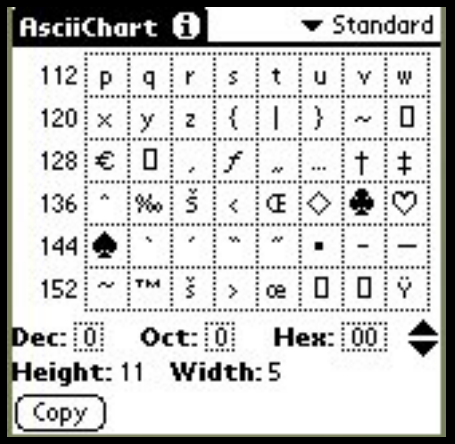
- Both character's width in pixels and size in bytes are dependent on encoding
- Localization can increase field width by 50% (either direction of translation)
- String buffers for fields need to be made longer as well
- Use FntWCharWidth for pixel width of a single WChar

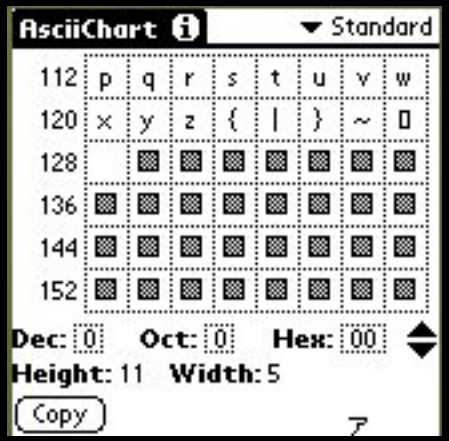




Character Encodings

- Latin, Shift-JIS, GBK, Big-5
- Shift-JIS defines 0x81 – 0x9F and 0xE0 – 0xEF as the first byte of a 2 byte character
  (2nd byte is 0x40 – 0x7E, 0x80 – 0xFC)
- 0-127 are the same for our Latin and Shift-JIS encodings, other than 0x5C (‘\’ vs. ¥)
- There are many other multi and double byte encoding schemes, including UTF-8 for Unicode

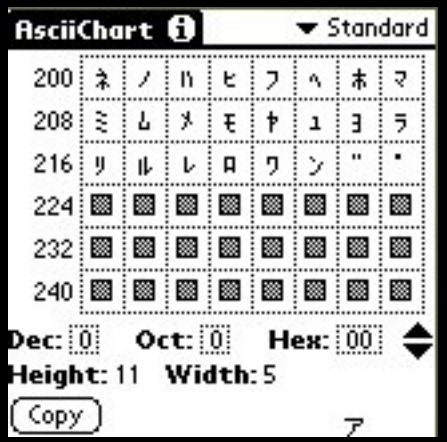




Character Encodings

- Roman letters and “katakana” exist both as single (in high ASCII) and double byte characters in Shift-JIS
- Fancy and European characters are in Latin High ASCII
- Only use High ASCII when you know it’s okay
- In Palm OS® 3.5 the Horizontal Ellipse and Numeric Space moved into control range 0x18 and 0x19
  - TxtGlueGetHorizEllipsisChar
  - TxtGlueGetNumericSpace





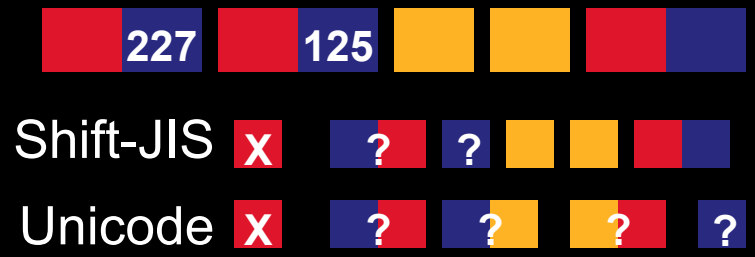
Character Encodings

- Our virtual characters are represented with 2 bytes, and have values > 255. Check the command bit in the keydown event record. Unicode might overlap this
- In general, use the Text Manager to work with any character encoding
- Use TxtGlueCharIsVirtual to check if you have a virtual character (hides Graffiti bug in older versions of Palm OS)

2 Byte Characters

Golden Rule: A Byte Is Not a Character

- Always use WChar type for local variables and function parameters (strings are still Char*)
- Avoid looking at 1 byte of a multi-byte character
- Make sure an offset into a string never references the 2nd byte of a double byte character. Use TxtCharBounds if necessary



227 can be 1st or 2nd byte in shift-jis

125 can be 2nd but not 1st in shift-jis



2 Byte Characters

- Use TxtCharIsXXX functions instead of old IsXXX macros (ex: TxtCharIsPrint not IsPrint)
- GetCharAttr will not work with double byte, or anything other than simple one byte encodings. Use TxtCharAttr or glue function instead
- Use TxtTransliterate for upper case and lower case transliteration. StrToLower calls TxtTransliterate.

Use Text Manager to Move Through Strings

- Use TxtGetNextChar and TxtGetPreviousChar to move through a string

```
width = 0;
offset = 0;
While(str[offset]){
 WChar theWChar;
 offset += TxtGetNextChar(str, offset,
    &theWChar);
 width += FntWCharWidth(theWChar);
}//for pixel width of string
```

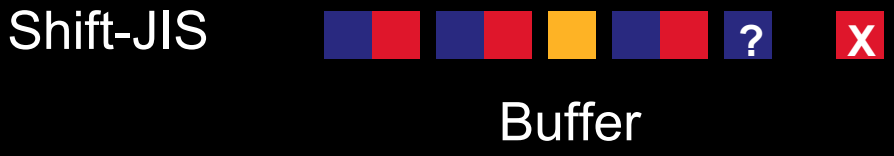


Don’t Slice Off the Second Half of a WChar

- Use TxtGetTruncationOffset to limit buffer lengths without splitting multi-byte char at end

```
buffer[TxtGetTruncationOffset(buffer,
  kMaxLen)] = '\0';
```

- Use WinDrawTruncChars to do visible truncation without splitting multi-byte char at end



Global Find



- Length of found text may not be the same as the length of search string, so use TxtFindString and stash length in custom parm

```
If (TxtFindString(stringToFind,
 params->strToFind, &pos, &matchLen)){
done = FindSaveMatch(findParams, recordNum,
 pos, fieldNum, matchLen,cardNum, dbID);
 //custom param
```

…then, in response to sysAppLaunchCmdGoTo

```
event.data.frmGoto.matchlen
 = gParam->matchCustom;
```

Flexible Base Resources

- Use TxtReplaceStr or TxtParamString to build strings at runtime
- Interface data belongs in resources
- Do not use custom resource types for text
- Use ‘tint’ resource for localizable numeric “soft” constants; sizes, widths, etc.

```
UInt32 ResLoadConstant(UInt16 rscID);
```

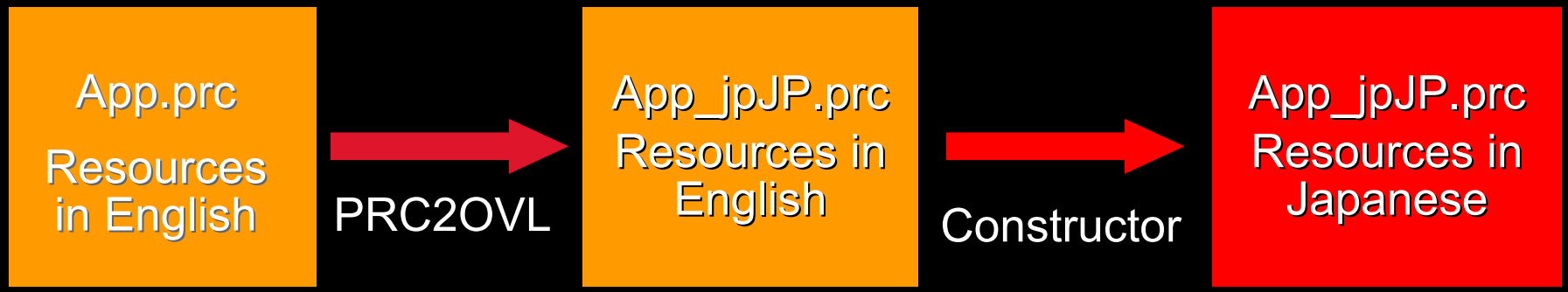
Making an Overlay

Method 1

- Start with Base PRC (any language)
- Create Overlay PRC with PRC2OVL Tool
- Edit resources in overlay with OnBoard RsrcEdit, PAR, Constructor, etc.
- This will insure that the overlay will match up with the base
- You have to recreate overlay if you alter non-code resources in base PRC

Making an Overlay

Method 1



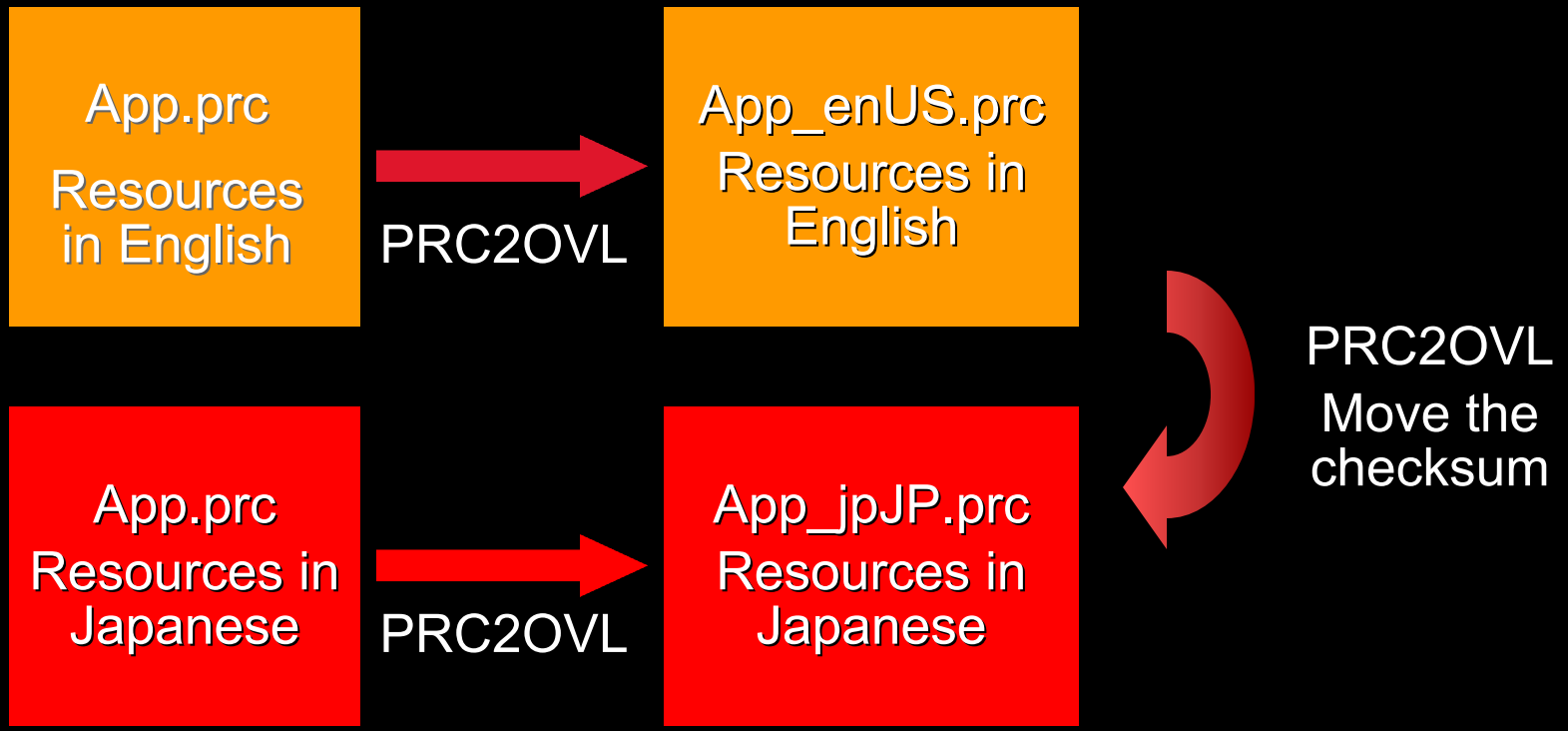
Making an Overlay

Method 2

- Create a Base Language PRC for each Language
- Choose 1 to be the shipping Base
- Create an Overlay from both PRCs
- Use the PRC2OVL tool's checksum modification to make them work (specifics)
- This makes it easier to change the base PRC and only that one part of the overlay
- You have to be careful to make sure that the UI matches on both!

Making an Overlay

Method 2



Coding for Overlays

Gotchas

- Be careful about what DB you are opening
  - DmNextOpenResDatabase will return the overlay database not the app
  - DmGet1Database searches both base and overlay
  - DmGetResource searches all open databases
  - Overlay DB can only be opened in read mode



Delivering Overlays to the Handheld

- Get country and language and install proper overlay
  - `SyncReadFeature('psys', 5 /*country*/, &result);`
  - `SyncReadFeature('psys', 6 /*language*/, &result);`
  - See PalmLocale.h to decipher result
- Bind overlays into application
  - Create Overlay
  - Add overlay file as custom resource
  - Expand overlay on launch and relaunch
  - Come to 461Lab for example…
- Trust the user to install the right one

Tool Links

- PRC2OVL
  - Comes with “Other Tools” from SDK
- OnBoard RsrcEdit
  - http://www.individeo.net/
- PRC Constructor
  - Will be in the new SDK :)
- PAR
  - http://www.djw.org/product/palm/par/index.html
- Multilizer
  - http://www.multilizer.com/solutions/tool/palm/
- ASCII Chart
  - http://www://astro.uchicago.edu/home/web/valdes/pilot/
- PowerGlot
  - http://www.powerglot.com



PalmSource®

Conference and Expo